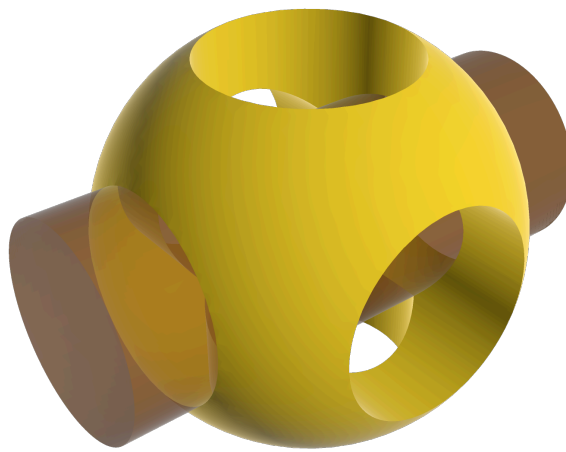




maquette-scad

Parametric CAD in Typst, via OpenSCAD + Manifold

typst.app/universe/package/maquette-scad
github.com/bernsteining/maquette
bernsteining.github.io/maquette

Version 0.1.0 · September 17, 2026

CONTENTS

Contents

Introduction	2
Where to find sample .scad files	2
Quickstart	2
Config Reference	3
Building geometry with scadypst	3
DSL reference	4
3D primitives	4
2D primitives (extrusion sources)	4
2D → 3D lifting (and 3D → 2D)	4
Transforms (2D or 3D)	4
Booleans and hulls	4
Plane operations	5
Colours and alpha	5
The scadypst() compile call	6
Name clashes with Typst built-ins	6
Compiling .scad sources with compile-scad	6
Inspection: scadypst-info / compile-scad-info	7
Decomposing: scadypst-parts / compile-scad-parts	7
Vertex reduction: simplify(child, epsilon)	7
Smooth shading: calculate-normals(child, sharp_angle)	8
Two ways to add smooth normals	8
Ray casting: scadypst-raycast / compile-scad-raycast	8
Direct 2D → SVG: scadypst-svg / compile-scad-svg	9
Multi-file projects: compile-scad-tree	9
What compile-scad accepts	10

Introduction

maquette-scad builds 3D models right inside your Typst document. You write the geometry, it compiles, and the render lands in your PDF. There is no OpenSCAD to install and nothing to run on the side.

There are two ways to describe a model. The first is the DSL: you build the shape from Typst helpers such as `cube`, `sphere`, `difference` and `translate`, and reach for Typst's own `for`, `range` and `calc` whenever the geometry is procedural. The second is to pass the text of an existing `.scad` file, libraries and all.

Either way you get a watertight mesh, which the Manifold kernel guarantees, and `maquette` renders it. `render-scad` does both steps in a single call. When a shape is flat, the `-svg` variants skip the rasterizer and write 2D vector output straight out.

You can try all of this in the browser at bernsteining.github.io/maquette. Edit the SCAD, orbit the result, adjust the render, and copy the Typst it generates.

This document is the API reference: what each helper takes, what the compile calls return, and how to hand in sidecar files. Camera, lighting and post-processing belong to `maquette`, so see its manual for those. For the OpenSCAD language itself, see the OpenSCAD Users Manual.

Where to find sample .scad files

The DSL examples in this document build their geometry inline, so you need no external source to follow along. If you want to try the `compile-scad(read("..."))` path, `.scad` files are easy to find:

- `openscad/openscad examples/` is the set that ships with the OpenSCAD editor.
- BOSL2 is a large utility library with hundreds of documented fragments.
- Thingiverse (openscad tag) is a community archive; many entries ship the `.scad` alongside the `.stl`.

Quickstart

`render-scad` compiles and renders in one call. Pass it a DSL tree or a `.scad` path, followed by any of `maquette`'s render options.

```
#import "@preview/maquette-scad:0.1.0": cube, difference, render-scad, sphere

#render-scad(
  difference(
    cube(20, center: true),
    sphere(12, fn: 48),
  ),
  camera: (40, 40, 40),
  up: (0, 0, 1),
)
```

render-scad also takes source text. It is the same call, with a .scad file in place of a tree:

```
#import "@preview/maquette-scad:0.1.0": render-scad

#render-scad(read("model.scad"))
```

One call compiles and renders; no maquette import. Examples below use a show-part wrapper so the code can focus on geometry.

Config Reference

render-scad takes a few compile options, then forwards every other named argument to maquette's render-ply. All are optional; defaults are shown.

```
{ // — Compile options (maquette-scad) —————
  "fn": 32, // Default $fn segment count; a per-primitive fn: overrides it
  "smooth-normals": 30, // Whole-mesh crease/sharp angle (deg) for smooth normals; none = faceted
                        // (same threshold as the calculate-normals tree-op's sharp_angle)
  "files": {}, // Multi-file .scad parts: { "path": bytes } (or pass a read: lambda)
  "bin": {}, // Binary meshes for import(): { "name": bytes }
  "font": null // TTF/OTF bytes for text(); null = bundled DejaVu Sans
}
```

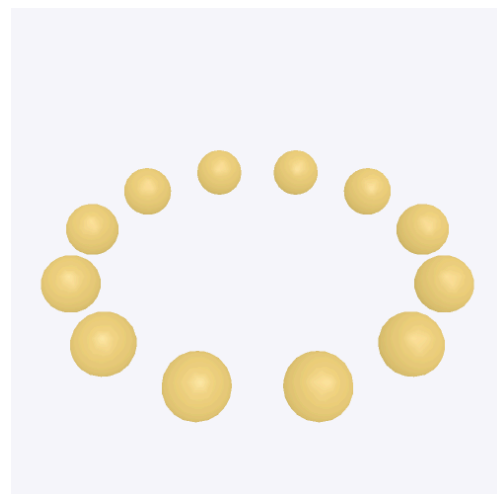
Every other argument forwards to maquette's render-ply: camera, azimuth, elevation, up, fov, zoom, background, color, shading, lights, shadows, ssao, and the rest. See the maquette manual for that full option surface. render-scad overrides two of its defaults to suit CAD facets: specular: 0 and cull_backface: false (matte and two-sided).

Building geometry with scadypst

scadypst(tree) walks a tree of Typst dicts and returns PLY bytes. Each helper such as cube, sphere or translate builds a node, and nothing runs until scadypst receives the whole tree.

Because the tree is Typst code, iteration and arithmetic come from Typst. Here a ..for spread inside union places twelve spheres on a ring:

```
#let ring = union(..for i in range(12) {
  let a = i * 30
  let r = 20
  (translate(
    (calc.cos(a * 1deg) * r, calc.sin(a * 1deg) * r, 0),
    sphere(3, fn: 24),
  ),)
})
#show-part(scadypst(ring))
```



The variadic ops (union, difference, hull, intersection) accept ..items, so you can spread a Typst for block to feed them a computed list.

DSL reference

Every helper takes Typst-native named arguments, as in `cube(20, center: true)`. The signatures below list positional arguments first, then named ones. `fn: N` sets the segment count for that primitive.

3D primitives

Signature	Description
<code>cube(size, center: false)</code>	<code>size</code> is a number for a uniform box, or a 3-array (<code>x</code> , <code>y</code> , <code>z</code>).
<code>sphere(r, fn: none)</code>	A sphere of radius <code>r</code> .
<code>cylinder(h, r: none, r1: none, r2: none, center: false, fn: none)</code>	Pass <code>r</code> for a straight cylinder, or <code>r1</code> and <code>r2</code> for a cone or frustum.
<code>polyhedron(points, faces)</code>	A raw mesh: <code>points</code> is <code>((x, y, z), ...)</code> , <code>faces</code> is <code>((i, j, k, ...), ...)</code> .

2D primitives (extrusion sources)

Signature	Description
<code>square(size, center: false)</code>	A rectangle; <code>size</code> is a number or <code>(w, h)</code> .
<code>circle(r, fn: none)</code>	A circle of radius <code>r</code> .
<code>ellipse(w, h, fn: none)</code>	An ellipse of width <code>w</code> and height <code>h</code> .
<code>polygon(points, paths: none)</code>	An arbitrary 2D shape. <code>paths</code> lists index-rings into <code>points</code> ; the first ring is the outer boundary, the rest are holes.
<code>ngon(sides, r, fn: none)</code>	A regular polygon.
<code>star(points, outer, inner)</code>	An <code>n</code> -pointed star.
<code>rounded-square(w, h, r, fn: none)</code>	A rectangle with radius- <code>r</code> corners.
<code>scad-text(str, size: 10)</code>	Glyph outlines from the font passed to <code>scadypst()</code> 's <code>font</code> : argument.
<code>import-mesh(file)</code>	References an STL or OBJ passed via <code>scadypst()</code> 's <code>bin</code> : dict.

2D → 3D lifting (and 3D → 2D)

Signature	Description
<code>linear-extrude(h, child, center: false, twist: 0, scale: 1, slices: none)</code>	Extrudes a 2D shape to height <code>h</code> . <code>twist</code> is degrees over the full height; <code>scale</code> is a taper from 0 to 1, or an <code>(sx, sy)</code> pair.
<code>rotate-extrude(child, angle: 360, fn: none)</code>	Revolves a 2D profile (living in the +x half-plane) around the z-axis.
<code>projection(child)</code>	Flattens a 3D solid to its 2D shadow on the Z=0 plane.

Transforms (2D or 3D)

Signature	Description
<code>translate(v, child)</code>	<code>v</code> is a 2- or 3-vector.
<code>rotate(deg, child)</code>	Euler degrees (<code>x</code> , <code>y</code> , <code>z</code>), or a single number for a 2D rotation.
<code>scale(v, child)</code>	Per-axis scale.
<code>mirror(v, child)</code>	Reflect across a plane whose normal is <code>v</code> .
<code>multmatrix(m, child)</code>	A 4×4 (or 4×3) affine matrix — the escape hatch for anything the named transforms cannot express.
<code>resize(v, child)</code>	Non-uniform scale to fit a target bounding box.
<code>offset(d, child)</code>	Grow (<code>d > 0</code>) or shrink (<code>d < 0</code>) a 2D shape by <code>d</code> .
<code>color(rgb, child, alpha: none)</code>	<code>rgb</code> is a 3-array of 0–1 floats, or a 4-array (<code>r</code> , <code>g</code> , <code>b</code> , <code>a</code>). See <i>Colours and alpha</i> below.

Booleans and hulls

Signature	Description
<code>union(..items)</code>	Combine every child into one solid.
<code>difference(..items)</code>	The first child minus all the rest.
<code>intersection(..items)</code>	Keep only the region shared by every child.
<code>hull(..items)</code>	Convex hull of the union (3D).
<code>hull-pts(points)</code>	Convex hull of a raw 3D point set — a list of (<code>x</code> , <code>y</code> , <code>z</code>) triples — when you have coordinates rather than geometry to wrap.

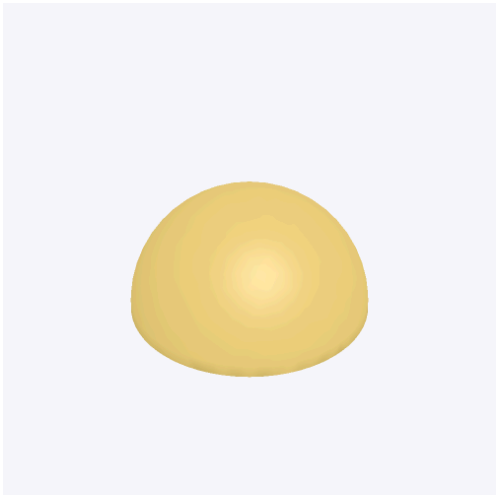
Signature	Description
minkowski(...items)	Minkowski sum (3D).

These are all variadic, so a ... spread over a Typst for block feeds them a computed list of children.

Plane operations

Signature	Description
slice(child, z: 0)	A horizontal cross-section of a 3D solid at the given Z, returned as a 2D shape (pipe into scadypst-svg(...)). Unlike projection(...), it does not union every slice.
trim(child, normal, offset: 0)	Cuts with a plane, keeping the half where dot(pos, normal) ≥ offset. Exact, and cheaper than the “difference() with a giant cube” trick.
simplify(child, epsilon: 0.01)	Douglas–Peucker vertex reduction for 2D and 3D. Its own section has a before/after count.
calculate-normals(child, sharp_angle: 60)	Attaches per-vertex normals for smooth shading; the crease-angle threshold keeps sharp edges crisp. 3D only, and visible only under shading: "smooth".

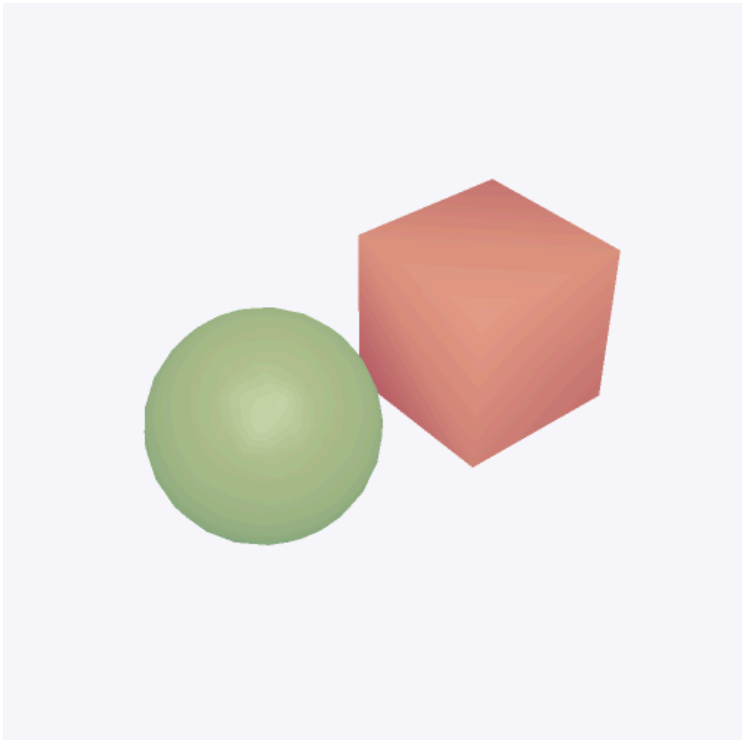
```
#show-part(scadypst(
  trim(sphere(10, fn: 64), (0, 0, 1))
))
```



Colours and alpha

Colours propagate through boolean operations. The emitted PLY carries per-vertex RGB and render-ply picks it up automatically. Any part without a color() wrapper inherits the render config’s color: argument.

```
#show-part(
  scadypst(union(
    color((0.9, 0.3, 0.3), cube(10, center: true)),
    color((0.3, 0.7, 0.4), translate((15, 0, 0), sphere(6, fn:
32))),
  )),
  color: none,
)
```



Alpha is supported. Two equivalent forms:

```
color((0.9, 0.35, 0.35, 0.5), child)           // RGBA in one 4-array
color((0.9, 0.35, 0.35), child, alpha: 0.5)    // RGB + separate alpha
```

Interior geometry shows through translucent subtrees, so a coloured cover can reveal the shape's internal features.

The `scadypst()` compile call

```
scadypst(node, bin: (:), font: none, fn: 32)
```

- **node**: the tree returned by any DSL helper, usually a top-level union or difference, or a single primitive.
- **bin**: a dict of sidecar bytes referenced by `import-mesh(file)`. The keys match the string passed to `import-mesh`, and the values are `read("path", encoding: none)`.
- **font**: TTF or OTF bytes for `scad-text(...)`. One font per compile.
- **fn**: the default \$fn for the whole compile. A per-primitive `fn`: overrides it.

It returns the same PLY bytes that `compile-scad` returns, ready to pass to `render-ply`.

Name clashes with Typst built-ins

A `glob-import (#import "...": *)` shadows seven of Typst's own functions:

`scale`, `rotate`, `circle`, `square`, `ellipse`, `polygon`, `color`. (Additionally, the OpenSCAD text primitive is re-exported as `scad-text` to avoid shadowing `#text`.)

Two ways around it:

```
// Option 1: namespace the plugin
#import "@preview/maquette-scad:0.1.0"
#let part = maquette-scad.scadypst(
  maquette-scad.difference(
    maquette-scad.cube(20, center: true),
    maquette-scad.sphere(12, fn: 48),
  )
)

// Option 2: glob-import the plugin's `scad-*` aliases
#import "@preview/maquette-scad:0.1.0": scad-color, scad-scale, scad-rotate, scad-circle, scad-square,
scad-ellipse, scad-polygon
```

Both leave Typst's own `#text` / `#circle` / ... intact. The `scad-*` aliases live alongside the unprefixed names, so you can mix approaches within one document.

Compiling .scad sources with `compile-scad`

For a standalone .scad file, one call is enough:

```
#render-scad(read("part.scad"))
```

Real-world .scad files rarely stand alone: they use `/ include` a library, `import` an STL or OBJ for a fastener, or ship a font for embossed text. The wasm sandbox has no filesystem, so every sidecar is passed explicitly as bytes:

```
#compile-scad(read("main.scad"),
  files: (
    "utils.scad":      read("utils.scad"),
    "gears.scad":      read("gears.scad"),
    "MCAD/nuts_and_bolts.scad": read("MCAD/nuts_and_bolts.scad"),
  ),
  bin: (
    "mount.stl":      read("mount.stl", encoding: none),
  ),
  font: read("logo.ttf", encoding: none),
  fn: 48,
  trace: "trace.log",
)
```

Options:

- **source** (positional string): the .scad text, almost always read("path.scad").
- **files** (dict): resolves every use <name> and include <name> in the source. The keys are the exact names the .scad uses, and nested paths like MCAD/foo.scad are keys with slashes.
- **bin** (dict): sidecar mesh files referenced by import(). The keys match the import argument. STL and OBJ are recognised; DXF, 3MF and AMF are not.
- **font** (bytes): a TTF or OTF file used by text(). One font per compile.
- **fn** (int): the default \$fn for this compile. Any per-primitive fn: overrides it.
- **trace** (string): dumps the evaluator trace to this path. It helps when a nested for or module is not producing what you expected.

Inspection: scadypst-info / compile-scad-info

Returns a Typst dict with the final geometry's stats without building the PLY. Useful for laying parts out by their real size, annotating a document with computed volumes, or failing early if a compile produced empty geometry.

Fields: bbox_min, bbox_max, center, radius, volume, surface_area, num_tri, num_vert, genus.

```
#let info = scadypst-info(cube(20, center: true))
#raw(block: true, lang: "yaml",
  "volume:      " + str(info.volume) + " mm³\n" +
  "surface_area: " + str(info.surface_area) + " mm²\n" +
  "num_tri:      " + str(info.num_tri) + "\n" +
  "bbox_min:     " + repr(info.bbox_min) + "\n" +
  "bbox_max:     " + repr(info.bbox_max)
)
```

```
volume:      8000 mm³
surface_area: 2400 mm²
num_tri:      12
bbox_min:     (-10, -10, -10)
bbox_max:     (10, 10, 10)
```

compile-scad-info(src, files: (:), bin: (:), font: none, fn: 32) is the .scad-source variant with the same return shape.

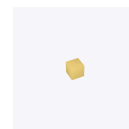
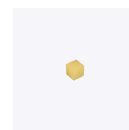
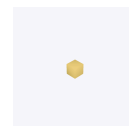
Decomposing: scadypst-parts / compile-scad-parts

Splits the final geometry into its connected components via `Manifold::decompose()` and returns each as its own PLY. The return type is array<bytes>; each element is renderable via `render-ply` in isolation.

Useful for laying out multiple pieces separately (laser-cut sheet layouts, exploded-view annotations, per-part BOM tables).

```
#let parts = scadypst-parts(union(
  translate((0, 0, 0), cube(4, center: true)),
  translate((10, 0, 0), cube(4, center: true)),
  translate((20, 0, 0), cube(4, center: true)),
))
Got #parts.len() parts:
#for p in parts {
  show-part(p, width: 25%)
  h(0.3em)
}
```

Got 3 parts:



compile-scad-parts(src, files: (:), bin: (:), font: none, fn: 32) is the .scad-source variant.

Vertex reduction: simplify(child, epsilon)

Collapses edges shorter than epsilon, measured in the input's units. It is a Douglas-Peucker style pass that strips micro-detail from booleans, or slims a mesh before you ship it through PLY or SVG.

It is dimension-agnostic: pass a 2D shape (CrossSection) or a 3D solid (Manifold), and the op dispatches on the child.

```
#let ring = union(..range(64).map(i => {
  let a = i * 360deg / 64
  translate((calc.cos(a) * 10, calc.sin(a) * 10),
    circle(1.2, fn: 8))
}))
```

```
before → num_tri: 1152
after  → num_tri: 112
```

```
#let ring-info = scadypst-info(ring)
#let simple-info = scadypst-info(simplify(ring, epsilon: 0.5))
#raw(block: true, lang: "yaml",
  "before → num_tri: " + str(ring-info.num_tri) + "\n" +
  "after → num_tri: " + str(simple-info.num_tri)
)
```

epsilon is in model units: keep it tiny for a fine mesh, and make it generous when you want to knock a curve down to a coarse polyline.

Smooth shading: calculate-normals(child, sharp_angle)

Attaches per-vertex normals to a 3D solid so maquette can smooth-shade curved surfaces while keeping crisp edges. Adjacent faces whose dihedral angle exceeds sharp_angle (degrees) stay sharp, and the rest blend. Run it after the final booleans and hulls, since those discard any existing normals.

The effect only shows under smooth shading. openscad-view is flat by design, so switch shading: "smooth" (and drop openscad-view) for the render call.

```
#let part = calculate-normals(
  difference(sphere(10, fn: 24), cube(12, center: true)),
  sharp_angle: 30,
)
#render-ply(scadypst(part),
  width: 3.5cm, shading: "smooth",
  azimuth: 30, elevation: 25)
```



Lower sharp_angle = more edges classed as sharp (crisp box corners). Higher = more edges smoothed (softer curves).

Two ways to add smooth normals

calculate-normals (above) is a **tree op**: it smooths whatever subtree it wraps. Use it when only one region of a bigger model should render smooth, say the sphere but not the cube parts.

The end-call helpers scadypst(..., smooth-normals: 30) and compile-scad(..., smooth-normals: 30) are the **whole-scene** shortcut: they run the same calculate_normals pass on the final mesh right before it's serialized. Same output, less typing when you want the entire model smoothed:

```
// tree-op form: smooths just the sphere
scadypst(union(
  calculate-normals(sphere(10, fn: 24), sharp_angle: 30),
  translate((15, 0, 0), cube(6, center: true)),
))

// end-call form: smooths everything
scadypst(my-model, smooth-normals: 30)
compile-scad(read("part.scad"), smooth-normals: 30)
```

Either way, the render call needs a smooth shading mode (shading: "smooth") to actually consume the normals.

Ray casting: scadypst-raycast / compile-scad-raycast

Fires a segment from origin to end at the evaluated geometry and returns an array of hits sorted by distance:

```
(
  ( face_id: <int>, distance: <float>,
    position: (x, y, z), normal: (x, y, z) ),
  ..
)
```

Use it to sample a terrain height under an XY coordinate, pick the face at a screen click, or check line-of-sight between two points. Empty array means the segment missed.

```
#let part = union(
  cube(10, center: true),
  translate((0, 0, 12), sphere(5, fn: 32)),
```

```
hits: 4
d=0.22 z=17
d=0.38 z=7
```

```

)
#let hits = scadypst-raycast(part,
  (0, 0, 30), (0, 0, -30))
#raw(block: true, lang: "yaml",
  "hits: " + str(hits.len()) + "\n" +
  hits.map(h =>
    "  d=" + str(calc.round(h.distance, digits: 2)) +
    "  z=" + str(calc.round(h.position.at(2), digits: 2))
  ).join("\n")
)

```

```

d=0.42  z=5
d=0.58  z=-5

```

`compile-scad-raycast(src, origin, end, files: (:), bin: (:), font: none, fn: 32)` is the `.scad-source` variant.

Direct 2D → SVG: `scadypst-svg` / `compile-scad-svg`

For sources that resolve to a 2D shape (circle, square, polygon, text, hulls of 2D things, offset, slice, projection), the plugin can emit an SVG document directly, with no maquette in the loop and no rasterizer. The result is resolution-independent, Inkscape-editable, and laser-cutter-ready.

Errors if the tree resolves to a 3D solid.

```

#image(scadypst-svg(difference(
  union(
    circle(10, fn: 64),
    translate((14, 0), square(20, center: true)),
  ),
  circle(4, fn: 48),
)), width: 65%)

```

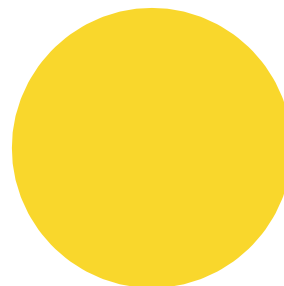


`slice(child, z: 0)` composes cleanly with `-svg`: 3D model → horizontal cross-section → vector contour, in one call.

```

#image(scadypst-svg(slice(sphere(10, fn: 64), z: 4)), width: 60%)

```



`compile-scad-svg(src, ...)` is the `.scad-source` variant.

Multi-file projects: `compile-scad-tree`

A project split across many files with use `<..>` and `include <..>` compiles in one call. `compile-scad-tree` reads the entry file and follows its include graph for you, so there is no file list to maintain:

```

#import "@preview/maquette-scad:0.1.0": openscad-view, render-scad-tree

#render-scad-tree(
  "Cyclone.scad",
  root: "cyclone-src/",
  read: p => read(p),
  ..openscad-view,
  azimuth: 35,
  elevation: 20,
  up: (0, 0, 1),
  background: none,
)

```

`root` is the source folder joined onto each discovered path, and `entry` is the top file inside it. Pass a `read: lambda` scoped to your document, `read: p => read(p)`, so the sources are read from **your** project; a package's own `read()` cannot reach your files. The whole Cyclone-PCB-Factory below, all 69 files, comes from that single call:

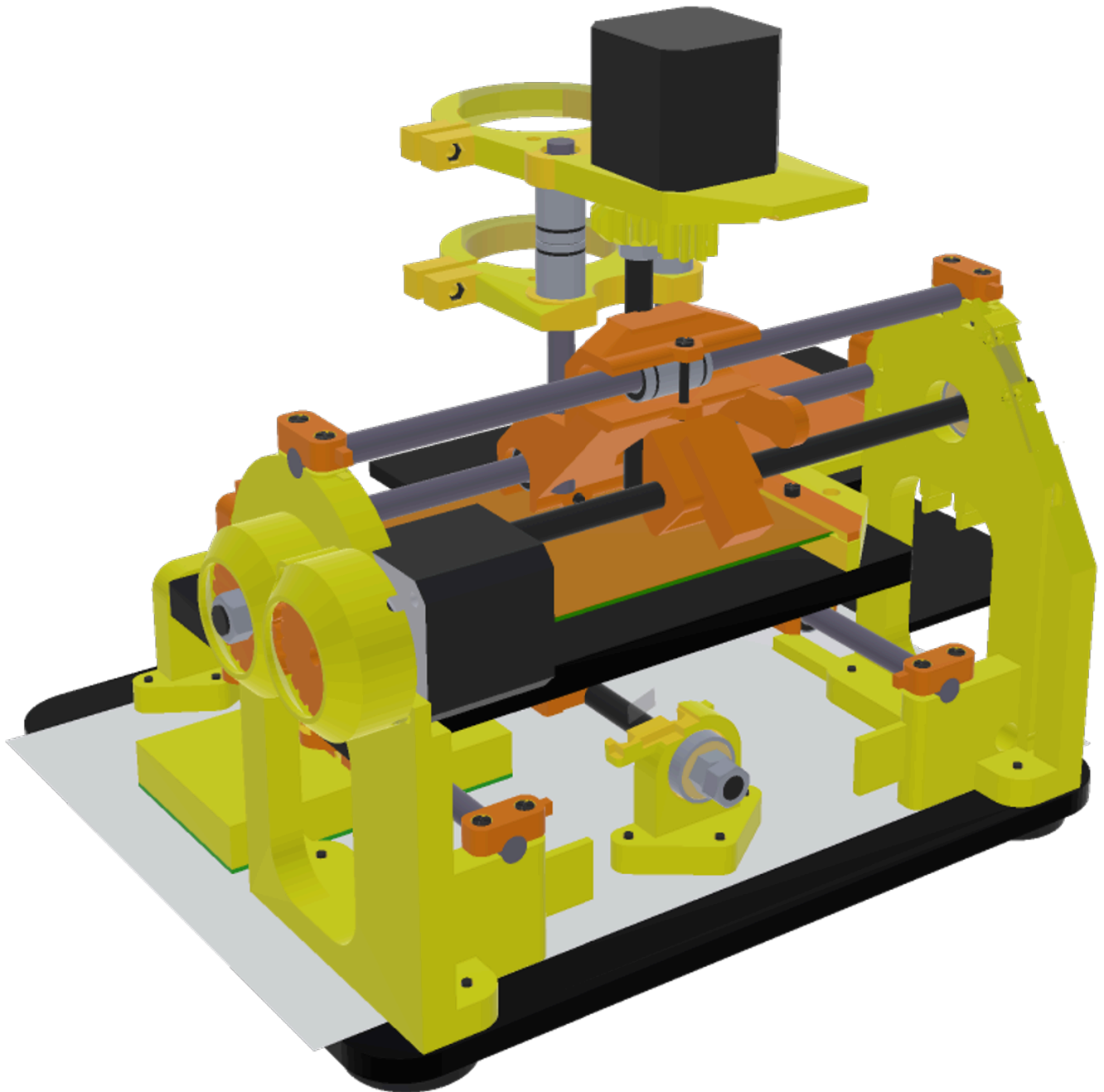


Figure 1: The full Cyclone-PCB-Factory from one `compile-scad-tree` call.

Need the files as a dict instead (to edit one before compiling)? `scad-collect(entry, root:)` returns `path → source` for `compile-scad(main, files: ...)`.

What `compile-scad` accepts

If your `.scad` source works in the OpenSCAD editor, it should work here; the evaluator covers everything a typical file uses. The known gaps are `surface()` (heightmap import), `import()` of DXF, the `$vpr` / `$vpt` / `$vpd` / `$vpf` viewport variables, and adaptive tessellation from `$fa` / `$fs` (we use `$fn` or the per-primitive `fn:` instead). Any external STL or OBJ mesh referenced by `import()` must be routed through the `bin:` argument, since the plugin has no filesystem access.

The exhaustive per-feature tracker lives at `crates/maquette-scad/FIDELITY.md`.

Two `wasm` modules cooperate. `maquette-scad.wasm` (this crate) compiles geometry to PLY. `maquette.wasm` renders it. They exchange nothing but the PLY blob, so you can also feed a `scadypst(...)` output into `maquette-gltf` (wrapped in a minimal `.gltf`) when you want PBR shading on procedural geometry.